Studio Nx

Backend(FastAPI/Python) 25 — SignupRequest · Nickname

2026-10-12

Backend(FastAPI/Python) 25 — 입력 스키마 `SignupRequest` · `Nickname`

항목	내용
파일	`studionx/backend/app/schemas/auth.py`, `studionx/backend/app/api/v1/auth.py`
작업	P1-04 회원가입: 가입 입력 형식 검사 (PRD 01 FR-101, P1-04 마지막 작업)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	24 `GET /auth/check-nickname` · 26 `create_access_token()` (P1-05)

1. 하는 일

가입 요청(22)과 닉네임 확인(24)의 **입력 모양**을 정합니다. 형식이 틀리면 라우터에 닿기 전에 FastAPI가 422로 돌려보냅니다.

칸	바꾸기 전 (템플릿)	바꾼 뒤
`email`	`str` (아무 문자열)	`EmailStr`: 이메일 형식을 검사하고, 앞뒤 공백을 지우며, 254자 이하만 받습니다
`nickname`	`str` 2~20자 (공백을 포함한 길이)	`Nickname`: 앞뒤 공백을 **먼저 지운 뒤** 2~20자인지 검사합니다

입력	바꾸기 전	바꾼 뒤
`nickname: " a "`	통과(5자)하고, 저장은 `"a"`(1자)로 됩니다	**422**입니다 (공백을 빼면 1자)
`nickname: " ab "`	통과하고, 서비스가 `"ab"`로 바꿉니다	통과하고, 스키마가 `"ab"`로 바꿉니다
`email: "kim@naver"`	통과해서 DB에 이상한 이메일이 들어갑니다	**422**
`email: " kim@naver.com "`	통과하고, 서비스가 공백을 지웁니다	통과하고, 스키마가 공백을 지웁니다

2. 언제 쓰나

쓰는 곳	사용 방식
22 `POST /auth/signup`	`data: SignupRequest`로 본문 JSON을 이 모양으로 검사합니다
24 `GET /auth/check-nickname`	`nickname: Annotated[Nickname, Query()]`로 주소의 `?nickname=`을 같은 타입으로 검사합니다
23 `GET /auth/check-email`	이미 `EmailStr`을 쓰고 있습니다 (23번에서 바꿨습니다)

확인과 가입이 같은 타입을 쓰는 이유: 중복 확인에서 "✓ Available"이라고 해 놓고 가입할 때 형식 오류가 나면 안 됩니다. 같은 타입을 쓰면 규칙이 한 곳에만 있게 됩니다.

3. 코드

import (`schemas/auth.py` 맨 위)

```
from datetime import datetime
from typing import Annotated

from pydantic import BaseModel, EmailStr, Field, StringConstraints
```

`Annotated`, `EmailStr`, `StringConstraints`를 이번에 추가했습니다.

import	가져오는 것	쓰는 곳
`Annotated` (파이썬 기본)	타입에 조건을 덧붙이는 도구	`Nickname`
`EmailStr`	이메일 형식 타입 (23번에서 설치한 `email-validator`가 검사합니다)	`email`
`StringConstraints`	글자 조건 묶음 (공백 제거, 길이)	`Nickname`

함수 (`schemas/auth.py`)

```
NICKNAME_MIN, NICKNAME_MAX = 2, 20

# 닉네임 입력 타입 — 앞뒤 공백을 먼저 지우고 길이를 잼 (가입 · 중복 확인이 함께 씀, 2026-10-07)
Nickname = Annotated[
    str,
    StringConstraints(strip_whitespace=True, min_length=NICKNAME_MIN, max_length=NICKNAME_MAX),
]


class SignupRequest(BaseModel):
    email: EmailStr = Field(max_length=255)  # 형식 검사 · 앞뒤 공백 제거 (email-validator)
    nickname: Nickname
    password: str = Field(min_length=1, max_length=100)  # 정확한 8~20자 검사는 check_password_rules
    privacy_agreed: bool
    privacy_version: str = Field(max_length=20)
```

(`api/v1/auth.py`의 24번 라우터)

```
    nickname: Annotated[Nickname, Query()],     # 바꾸기 전: Annotated[str, Query(min_length=…, max_lengt
h=…)]
```

24번에서 넣었던 `min_length`와 `max_length`는 `Nickname` 안으로 들어갔기 때문에, 라우터의 `NICKNAME_MIN`과 `NICKNAME_MAX` import는 지웠습니다.

문법 풀이

import 문법	뜻
`from pydantic import BaseModel, EmailStr, Field, StringConstraints`	pydantic에서 네 이름을 한 줄로 가져옵니다

식	뜻
`Nickname = Annotated[str, StringConstraints(…)]`	**타입에 이름을 붙입니다**. "조건이 붙은 문자열"을 `Nickname` 한 단어로 여러 곳에서 씁니다
`strip_whitespace=True`	검사하기 **전에** 앞뒤 공백을 지웁니다. 지운 값이 그대로 `data.nickname`에 들어갑니다
`min_length`, `max_length`	공백을 지운 **뒤**의 길이입니다

식	뜻
`EmailStr = Field(max_length=255)`	이메일 형식과 255자 이하를 함께 검사합니다. 실제로는 이메일 규격의 상한인 254자에서 먼저 막힙니다
`Annotated[Nickname, Query()]`	같은 타입을 주소 값(`?nickname=`)에도 씁니다

4. 흐름 (요청이 들어올 때)

1. FastAPI가 JSON과 주소 값을 꺼냅니다.
2. `email`은 앞뒤 공백을 지운 뒤 형식을 검사하고, `nickname`은 앞뒤 공백을 지운 뒤 2~20자인지 검사합니다.
3. 하나라도 틀리면 02번 처리기가 422와 `details`(어느 칸이 틀렸는지)로 응답합니다.
4. 모두 맞으면 정리된 값(공백을 지운 이메일과 닉네임)이 라우터를 거쳐 서비스(21)로 갑니다.

5. 왜 이렇게

선택	이유
형식 검사는 스키마(입구)에서 함	틀린 입력이 서비스나 DB까지 가지 않습니다. 오류 모양도 다른 입력 오류와 같아집니다 (02)
비밀번호는 스키마에서 1~100자만 검사	8~20자와 조건 검사는 14번이 합니다. 조건별 실패 목록(`["upper", …]`)을 화면에 돌려주려면 서비스에서 검사해야 합니다
이메일 소문자 변환은 스키마가 아니라 17번에서	로그인(P1-05)과 중복 확인도 17번을 거칩니다. 비교 기준을 한 함수에 모아 둡니다
서비스(21)의 `strip()`과 `normalize_email()`은 그대로 둠	스키마를 거치지 않고 서비스를 부르는 곳(시험, 나중의 관리 기능)이 있어도 같은 기준이 지켜집니다
`Nickname`을 `schemas/auth.py`에 둠	P1-08 회원 정보 수정(`schemas/user.py`)에서도 닉네임을 바꿉니다. 그때 이 타입을 가져다 쓰면 됩니다

6. 시험

`tests/test_auth_schemas.py` (새 파일, 16개)

시험	확인한 것
닉네임 통과 4개	`beatmaker`, `" beatmaker "` → `beatmaker`, `" ab "` → `ab`, 20자
닉네임 실패 5개	1자, `" a "`, 공백만, 21자, 공백을 빼도 21자는 모두 `nickname` 오류입니다
이메일 공백	`" kim@naver.com "`은 `kim@naver.com`이 됩니다
이메일 실패 5개	`abc`, `a b@x.com`, `kim@naver`, `deleted+85@withdrawn.invalid`, 빈 문자열은 모두 `email` 오류입니다
이메일 길이	254자는 통과하고, 300자가 넘으면 `email` 오류입니다

`tests/test_auth_routes.py`에 3개 추가 (모두 27개)

시험	확인한 것
`GET /check-nickname?nickname= a`	422입니다 (24번에서 남아 있던 차이가 고쳐졌습니다)
`POST /signup` 닉네임 `" a "`	422이고, `details`에 `nickname`이 담깁니다
`POST /signup` 이메일 `kim@naver`	422이고, `details`에 `email`이 담깁니다

전체 시험 186개가 통과했고, `ruff` 검사도 통과했습니다.